

Curso de React

Apuntes teóricos y guía práctica

Elaborado por: Lihúén Villarreal

Índice de contenidos

Introducción.....	4
React.....	4
SPA vs MPA.....	4
UI declarativa.....	4
Componentes como funciones.....	5
Flujo de datos unidireccional.....	5
Virtual DOM.....	6
Configuración y entorno.....	6
Create React App.....	6
Vite (estructura típica).....	6
Estructura de un proyecto React.....	7
Archivos clave.....	7
index.html.....	7
main.jsx.....	8
App.jsx.....	8
JSX.....	8
Expresiones en JSX.....	8
Condicionales en JSX.....	9
Operador &&.....	9
Operador ternario.....	9
Listas y map.....	9
key: para qué sirve y buenas prácticas.....	10
Fragmentos (<> </>).....	10
Componentes.....	11
Componentes funcionales.....	11
Hooks.....	12
Props.....	12
Props vs estado.....	13
Destructuring de props.....	13
Default props.....	13
children.....	14
Estado y reactividad.....	14
useState.....	14
Estado primitivo vs objetos / arrays.....	15
Actualización inmutable.....	15
Función updater.....	16
Renderizado y re-render.....	16
Estado derivado.....	17

Manejo de eventos.....	17
Eventos sintéticos.....	17
Pasar parámetros a handlers.....	18
Formularios controlados.....	18
Inputs, selects y checkboxes.....	19
Inputs de texto.....	19
Selects.....	19
Checkboxes.....	19
Prevención de comportamiento por defecto.....	19
Ciclo de vida y efectos.....	20
useEffect.....	20
Dependencias.....	20
Efectos de montaje y desmontaje.....	21
Montaje.....	21
Desmontaje.....	21
Limpieza (cleanup).....	21
Errores comunes con useEffect.....	22
Comunicación entre componentes.....	22
Parent → child (props).....	22
Child → parent (callbacks).....	23
Levantar estado (lifting state up).....	23
Estado compartido.....	24
Renderizado condicional y composición.....	24
Renderizado condicional.....	24
Composición vs. herencia.....	24
Patrones comunes de composición.....	25
Componentes “contenedor” y “presentacional”.....	25
Manejo de listas y datos.....	26
Renderizado de listas.....	26
Filtrado y ordenamiento.....	26
Datos asincrónicos.....	26
Loading y error states.....	27
Fetching y asincronía.....	27
fetch / axios.....	27
Peticiones en useEffect.....	28
Cancelación de requests.....	28
Estados de carga.....	29
Buenas prácticas básicas.....	29
Hooks adicionales más usados.....	29
useRef.....	29
useMemo.....	30
useCallback.....	30
Cuándo NO usarlos.....	31
Context y estado global.....	31
Context API.....	31
Cuándo usar context.....	31
Provider y consumer.....	32
Problemas de performance.....	32

Alternativas.....	32
Estilos en React.....	33
CSS tradicional.....	33
CSS Modules.....	33
Styled-components.....	33
Inline styles.....	34
Convenciones comunes.....	34
Routing.....	34
Rutas.....	35
Links.....	36
Params.....	36
Rutas protegidas.....	36
Buenas prácticas y patrones.....	37
Convenciones para nombres.....	38
Performance.....	38
Memoización.....	38
Lazy loading.....	38
Code splitting.....	38
Cuándo optimizar.....	39
Errores comunes y debugging.....	39
Infinite re-renders.....	39
Dependencias mal definidas.....	39
Keys incorrectas.....	39
Mutación de estado.....	40
Uso incorrecto de hooks.....	40
Ecosistema React.....	40
React vs. frameworks (Next, Remix).....	40
Estado global (Redux, Zustand).....	41
UI libraries.....	41
Herramientas habituales.....	41
Anexo: Guía práctica para crear una aplicación en React.....	41
Crear el proyecto.....	42
Ejecutar el proyecto.....	42
Entender la estructura inicial.....	43
Limpiar el proyecto base.....	43
Crear estructura de carpetas básica.....	43
Crear un componente simple.....	43
Usar props y estado.....	44
Manejar eventos.....	45
Renderizar listas.....	45
Fetching de datos.....	47
Manejar estilos.....	49
Agregar routing.....	49
Verificar y ordenar.....	51

Introducción

React

React es una **librería de JavaScript para construir interfaces de usuario (UI)**. Su objetivo principal es facilitar la creación de UIs **complejas, dinámicas y mantenibles**, donde la interfaz cambia en función del estado de la aplicación.

El problema que React apunta a resolver es:

- Cuando una aplicación crece, **mantener sincronizado el estado de los datos con el DOM se vuelve difícil y propenso a errores**.
- Manipular el DOM de forma imperativa (*“busca este elemento, cámbiale esto, luego aquello”*) no escala bien.

React propone un enfoque distinto:

La UI es una función del estado.

En lugar de decir *cómo* modificar la interfaz paso a paso, se describe *cómo debería verse* para un determinado estado, y React se encarga de actualizarla.

SPA vs MPA

MPA (Multi Page Application):

- Cada acción importante implica cargar una nueva página desde el servidor.
- El servidor genera el HTML completo.
- Ejemplo clásico: sitios tradicionales con múltiples recargas.

SPA (Single Page Application):

- Se carga una única página HTML.
- La navegación y los cambios de vista ocurren en el navegador.
- JavaScript controla qué se muestra.

React se usa principalmente para construir **SPAs**, donde:

- La experiencia es más fluida.
- La UI puede reaccionar inmediatamente a cambios de estado.
- Se separa claramente frontend y backend.

UI declarativa

React utiliza un **modelo declarativo** para definir interfaces.

- **Imperativo:** dices *qué pasos ejecutar*.

- **Declarativo:** describes *el resultado esperado*.

En React:

- Se describe cómo se ve la UI para un estado dado.
- Cuando el estado cambia, React vuelve a evaluar esa descripción.

Esto reduce:

- Estados inconsistentes
- Lógica de sincronización manual
- Código difícil de razonar

Componentes como funciones

En React moderno, los componentes son **funciones de JavaScript**.

Un componente:

- Recibe datos (props)
- Devuelve una descripción de la UI (JSX)

Esto favorece:

- Reutilización
- Composición
- Testeo
- Lectura del código

Flujo de datos unidireccional

React impone un **flujo de datos de arriba hacia abajo**:

- Los datos bajan de componentes padres a hijos mediante props.
- Los hijos no modifican directamente el estado del padre.

Si un hijo necesita “comunicar algo hacia arriba”:

- El padre pasa una función.
- El hijo la ejecuta.

Este modelo:

- Hace el flujo de datos predecible
- Facilita el debugging
- Reduce efectos colaterales

Virtual DOM

React no modifica el DOM real directamente en cada cambio.

En su lugar:

- Mantiene una representación en memoria de la UI (Virtual DOM).
- Cuando el estado cambia, genera una nueva versión.
- Compara la versión anterior con la nueva.
- Aplica **solo los cambios necesarios** al DOM real.

Ideas importantes:

- El Virtual DOM **no es el objetivo**, sino un medio.
- React es eficiente porque **minimiza operaciones costosas sobre el DOM**.
- Como usuario de React, no se interactúa directamente con el Virtual DOM.

Configuración y entorno

Create React App

Create React App (CRA) es una herramienta oficial que permite **crear rápidamente un proyecto React preconfigurado**.

Proporciona:

- Configuración automática de Webpack, Babel y otras herramientas
- Un entorno de desarrollo listo para usar
- Convenciones claras y estables

Se suele usar cuando:

- Se quiere empezar rápido sin pensar en configuración
- Se está aprendiendo React
- No se necesita un setup personalizado

Actualmente, su uso es cada vez menor frente a alternativas más modernas, pero sigue siendo útil para **proyectos simples o educativos**.

Vite (estructura típica)

Vite es una herramienta moderna de desarrollo frontend que se usa ampliamente con React.

Ventajas principales:

- Arranque del servidor muy rápido

- Hot Module Replacement (HMR) eficiente
- Configuración mínima

Estructura típica de un proyecto React con Vite:

- `index.html`
- `src/`
 - `main.jsx`
 - `App.jsx`
 - otros componentes

Vite se encarga del bundling (empaquetado de archivos) y del entorno de desarrollo, manteniendo una estructura simple y explícita.

Estructura de un proyecto React

Aunque puede variar, un proyecto React suele organizarse de la siguiente forma:

- `index.html`: punto de entrada del HTML
- `src/`: código fuente de la aplicación
 - componentes
 - estilos
 - lógica

Dentro de `src`:

- Se colocan los componentes React
- Se separa la lógica por responsabilidades
- Se evita lógica compleja en el punto de entrada

No existe una única estructura correcta, pero se prioriza:

- Claridad
- Escalabilidad
- Mantenimiento

Archivos clave

`index.html`

- Contiene el contenedor raíz de la aplicación
- Suele tener un `div` donde se monta React
- No se modifica frecuentemente

main.jsx

- Punto de entrada de JavaScript
- Se inicializa React
- Se monta el componente raíz en el DOM

Aquí se conecta React con el HTML.

App.jsx

- Componente raíz de la aplicación
- Contiene la estructura principal de la UI
- Se compone a partir de otros componentes

Desde aquí se organiza el resto de la aplicación.

JSX

JSX es una **extensión de sintaxis** que permite escribir estructuras similares a HTML dentro de JavaScript.

Características clave:

- No es HTML real
- Se transforma a JavaScript estándar
- Permite incrustar expresiones JavaScript

JSX:

- Describe la UI
- Se evalúa como expresiones

JSX **no es**:

- Un template string
- Una mezcla directa de HTML y JavaScript
- Algo que el navegador entienda sin compilación

El objetivo de JSX es hacer que la definición de la UI sea más legible y expresiva dentro del código.

Expresiones en JSX

En JSX se pueden incrustar **expresiones de JavaScript** usando llaves `{}`.

- Dentro de `{}` se permite cualquier expresión válida de JavaScript
- No se permiten sentencias (`if`, `for`, `while`, etc.)

Ejemplos de expresiones válidas:

- Variables
- Operadores
- Llamadas a funciones
- Expresiones ternarias

Ejemplo:

```
const name = "Lihuén";

<h1>Hola {name}</h1>
<h2>{2 + 2}</h2>
<h3>{getTitle()}</h3>
```

JSX se evalúa siempre como una **expresión**, no como un bloque de instrucciones.

Condicionales en JSX

Como no se pueden usar `if` directamente, se utilizan patrones basados en expresiones.

Operador `&&`

Se usa cuando se quiere renderizar algo **solo si la condición es verdadera**.

```
{isLoggedIn && <UserProfile />}
```

Si la condición es falsa, no se renderiza nada.

Operador ternario

Se usa cuando se quiere elegir entre **dos opciones**.

```
{isLoading ? <Spinner /> : <Content />}
```

Es útil para manejar estados excluyentes.

Listas y map

Para renderizar listas se utiliza `Array.map()`.

- Se transforma un array de datos en un array de elementos JSX
- Cada iteración devuelve un elemento renderizable

```
const items = ["uno", "dos", "tres"];
```

```
<ul>
  {items.map(item => (
    <li>{item}</li>
  ))}
</ul>
```

Este patrón es central en React para trabajar con datos dinámicos.

key: para qué sirve y buenas prácticas

Cuando se renderizan listas, cada elemento debe tener una **prop key**.

La **key**:

- Identifica de forma única a cada elemento
- Permite que React detecte cambios, inserciones y eliminaciones
- Mejora la eficiencia del renderizado

Ejemplo:

```
{items.map(item => (
  <li key={item.id}>{item.name}</li>
))}
```

Buenas prácticas:

- Usar identificadores únicos y estables
- Evitar usar el índice del array como **key** (salvo casos muy puntuales)
- No reutilizar keys entre elementos distintos

Fragmentos (<> </>)

Un componente React debe devolver **un único nodo raíz**.

Los fragmentos permiten agrupar múltiples elementos sin agregar nodos extra al DOM.

Forma corta:

```
<>
  <h1>Título</h1>
  <p>Contenido</p>
</>
```

Forma explícita:

```
<React.Fragment>
  <h1>Título</h1>
  <p>Contenido</p>
</React.Fragment>
```

Los fragmentos:

- No generan HTML adicional
- Ayudan a mantener un DOM más limpio
- Son comunes en componentes simples

Componentes

Componentes funcionales

En React moderno, los componentes se definen principalmente como **funciones de JavaScript**.

Un componente funcional:

- Es una función
- Puede recibir props como argumento
- Devuelve JSX

Ejemplo:

```
function Greeting() {
  return <h1>Hola</h1>;
}
```

También se pueden definir como funciones flecha:

```
const Greeting = () => {
  return <h1>Hola</h1>;
};
```

Los componentes funcionales:

- Son más simples que los componentes de clase
- Permiten usar hooks
- Son el estándar actual en React

Hooks

En React, un **hook** es una función especial que permite usar características de React (como el estado o el ciclo de vida) dentro de **componentes funcionales**, sin necesidad de usar **clases**.

Antes de los hooks, para manejar **estado o efectos** se usaban **componentes de clase**. Los hooks permiten:

- Manejar **estado** (`useState`)
- Ejecutar **efectos secundarios** (`useEffect`)
- Compartir lógica entre componentes
- Escribir componentes más simples y reutilizables

Reglas importantes de los hooks:

1. Solo se llaman en el **nivel superior** del componente (no dentro de condicionales, bucles, ni funciones anidadas)
2. Solo se usan en **componentes funcionales** o en otros **hooks**
3. Su **nombre** siempre empieza con `use`

Hooks más comunes:

- `useState`
- `useEffect`
- `useContext`
- `useRef`
- `useMemo`
- `useCallback`

También se pueden crear hooks personalizados:

```
function useContadorInicial(valorInicial) {  
  const [valor, setValor] = useState(valorInicial);  
  return [valor, setValor];  
}
```

Props

Las **props** son datos que se pasan a un componente desde su componente padre.

Características de las props:

- Son de solo lectura
- Permiten configurar y reutilizar componentes
- Fluyen de arriba hacia abajo

Ejemplo:

```
function Greeting(props) {  
  return <h1>Hola {props.name}</h1>;  
}
```

```
<Greeting name="Lihuén" />
```

Las props permiten que un mismo componente se use con distintos datos.

Props vs estado

Aunque ambos influyen en lo que se renderiza, cumplen roles distintos.

Props:

- Se reciben desde afuera
- No se modifican dentro del componente
- Representan datos controlados por el padre

Estado:

- Se define dentro del componente
- Puede cambiar con el tiempo
- Representa datos propios del componente

Destructuring de props

Para mejorar la legibilidad, las props suelen desestructurarse.

→ En la firma de la función:

```
function Greeting({ name }) {  
  return <h1>Hola {name}</h1>;  
}
```

→ O dentro del cuerpo del componente:

```
function Greeting(props) {  
  const { name } = props;  
  return <h1>Hola {name}</h1>;  
}
```

Este patrón reduce repetición y hace el código más claro.

Default props

Los valores por defecto para props se definen actualmente usando **valores por defecto en la desestructuración**.

Ejemplo:

```
function Greeting({ name = "Mundo" }) {  
  return <h1>Hola {name}</h1>;  
}
```

Este enfoque:

- Reemplaza el uso de `defaultProps` en componentes funcionales
- Es más simple y explícito
- Se alinea con JavaScript moderno

children

`children` es una prop especial que representa el contenido incluido entre las etiquetas de un componente.

Ejemplo:

```
function Card({ children }) {  
  return <div className="card">{children}</div>;  
}  
  
<Card>  
  <p>Contenido interno</p>  
</Card>
```

`children` permite:

- Crear componentes contenedores
- Componer interfaces de forma flexible
- Evitar props rígidas

Estado y reactividad

useState

`useState` es el hook básico para agregar **estado** a un componente funcional.

- Permite declarar una variable de estado
- Proporciona una función para actualizarla
- Al cambiar el estado, se dispara un re-render

Ejemplo:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

useState:

- Se llama siempre en el nivel superior del componente
- Mantiene su valor entre renders

Estado primitivo vs objetos / arrays

El estado puede ser:

- **Primitivo:** número, string, boolean
- **Compuesto:** objetos o arrays

Ejemplo de estado primitivo:

```
const [isOpen, setIsOpen] = useState(false);
```

Ejemplo de estado como objeto:

```
const [user, setUser] = useState({ name: "Ana", age: 30 });
```

Ejemplo de estado como array:

```
const [items, setItems] = useState([]);
```

Cuando se usan objetos o arrays, se debe tener especial cuidado al actualizarlos.

Actualización inmutable

El estado **no se modifica directamente**. En su lugar, se crea una nueva versión.

Incorrecto:

```
user.age = 31;
```

```
setUser(user);
```

Correcto:

```
setUser({  
  ...user,  
  age: 31,  
});
```

Con arrays:

```
setItems([...items, newItem]);
```

La inmutabilidad:

- Permite que React detecte cambios
- Evita efectos colaterales
- Hace el comportamiento más predecible

Función updater

Cuando el nuevo estado depende del estado anterior, se usa la **función updater**.

Ejemplo:

```
setCount(prevCount => prevCount + 1);
```

Este patrón:

- Evita problemas con estados desactualizados
- Es especialmente importante en actualizaciones múltiples

Regla práctica:

- Si el nuevo estado usa el anterior, se usa la función updater

Renderizado y re-render

Un componente se renderiza:

- Cuando se monta por primera vez
- Cada vez que cambia su estado
- Cuando cambian sus props

En un re-render:

- Se vuelve a ejecutar la función del componente
- Se evalúa nuevamente el JSX

- React actualiza el DOM solo si es necesario

Un re-render:

- No implica recargar la página
- No implica recrear el DOM completo

Estado derivado

El **estado derivado** es aquel que puede calcularse a partir de props u otro estado.

Ejemplo **a evitar**:

```
const [fullName, setFullName] = useState("");
```

Si `fullName` puede obtenerse de `firstName` y `lastName`, no debería ser estado.

En su lugar:

```
const fullName = `${firstName} ${lastName}`;
```

Evitar estado derivado:

- Reduce duplicación
- Evita inconsistencias
- Simplifica la lógica

Regla general:

- Si algo se puede calcular, no debería almacenarse en estado

Manejo de eventos

Eventos sintéticos

React utiliza **eventos sintéticos**, que son una abstracción sobre los eventos nativos del navegador.

Características principales:

- Funcionan de forma consistente entre navegadores
- Tienen una API similar a los eventos nativos
- Se pasan como argumento al handler

Ejemplo:

```
function Button() {
```

```

    function handleClick(event) {
      console.log(event);
    }

    return <button onClick={handleClick}>Click</button>;
  }

```

Para la mayoría de los casos, se manejan igual que los eventos del DOM.

Pasar parámetros a handlers

Los handlers de eventos se pasan como **referencias a funciones**, no como llamadas.

Incorrecto:

```

<button onClick={handleClick(param)}>Click</button>

```

Correcto, usando una función flecha:

```

<button onClick={() => handleClick(param)}>Click</button>

```

De esta forma:

- La función se ejecuta solo cuando ocurre el evento
- Se pueden pasar parámetros personalizados

Formularios controlados

En React se suelen usar **formularios controlados**, donde el valor del input está ligado al estado.

Características:

- El estado es la única fuente de verdad
- Cada cambio actualiza el estado
- El valor del input se define desde React

Ejemplo básico:

```

const [name, setName] = useState("");

<input
  value={name}
  onChange={e => setName(e.target.value)}
/>

```

Este patrón permite:

- Validación
- Control total del formulario
- Sincronización clara entre UI y datos

Inputs, selects y checkboxes

Inputs de texto

- Se usa `value`
- Se actualiza con `onChange`

```
<input value={text} onChange={e => setText(e.target.value)} />
```

Selects

- Se usa `value` para la opción seleccionada

```
<select value={role} onChange={e => setRole(e.target.value)}>
  <option value="admin">Admin</option>
  <option value="user">User</option>
</select>
```

Checkboxes

- Se usa `checked` en lugar de `value`

```
<input
  type="checkbox"
  checked={isActive}
  onChange={e => setIsActive(e.target.checked)}
/>
```

Prevención de comportamiento por defecto

Algunos eventos tienen comportamientos por defecto del navegador (por ejemplo, envío de formularios).

Para prevenirlos se usa `event.preventDefault()`.

Ejemplo:

```
function handleSubmit(event) {
  event.preventDefault();
}
```

```
    // lógica de envío
  }

  <form onSubmit={handleSubmit}>
    <button type="submit">Enviar</button>
  </form>
```

Esto permite manejar la lógica completamente desde React sin recargar la página.

Ciclo de vida y efectos

useEffect

`useEffect` es el hook que permite ejecutar **efectos secundarios** en componentes funcionales.

Se consideran efectos secundarios, por ejemplo:

- Acceso a APIs
- Suscripciones
- Timers
- Manipulación de recursos externos al render

`useEffect` se ejecuta **después del renderizado**.

Ejemplo básico:

```
import { useEffect } from "react";

useEffect(() => {
  console.log("Efecto ejecutado");
});
```

Dependencias

`useEffect` recibe un segundo argumento: el **array de dependencias**.

Este array indica **cuándo debe volver a ejecutarse el efecto**.

- Sin array de dependencias: el efecto se ejecuta en cada render
- Array vacío (`[]`): el efecto se ejecuta solo al montar el componente
- Con dependencias: el efecto se ejecuta cuando alguna dependencia cambia

Ejemplo:

```
useEffect(() => {  
  fetchData(userId);  
}, [userId]);
```

La regla general:

- Toda variable usada dentro del efecto y definida fuera debería estar en el array

Efectos de montaje y desmontaje

Montaje

Para ejecutar un efecto solo al montar el componente, se usa un array de dependencias vacío.

```
useEffect(() => {  
  console.log("Componente montado");  
}, []);
```

Desmontaje

Para ejecutar lógica al desmontar el componente, el efecto debe **retornar una función**.

```
useEffect(() => {  
  return () => {  
    console.log("Componente desmontado");  
  };  
}, []);
```

Esta función se ejecuta:

- Al desmontar el componente
- Antes de volver a ejecutar el efecto

Limpieza (cleanup)

La función de cleanup se usa para:

- Cancelar suscripciones
- Limpiar timers
- Evitar memory leaks

Ejemplo:

```
useEffect(() => {  
  const id = setInterval(() => {
```

```
        console.log("tick");
    }, 1000);

    return () => clearInterval(id);
}, []);
```

La limpieza es especialmente importante en efectos que interactúan con recursos externos.

Errores comunes con useEffect

Errores frecuentes:

- Omitir dependencias necesarias
- Usar el efecto para lógica que no es un efecto secundario
- Generar loops infinitos por dependencias mal definidas
- Usar `useEffect` para derivar estado innecesario

Regla práctica:

- `useEffect` no es para sincronizar estados internos
- `useEffect` es para sincronizar React con el mundo exterior

Comunicación entre componentes

Parent → child (props)

La forma principal de comunicación en React es **de padre a hijo**, mediante props.

- Un componente padre pasa datos a un componente hijo
- El hijo recibe esos datos como argumentos
- El flujo es unidireccional

Ejemplo:

```
function Parent() {
    return <Child message="Hola" />;
}

function Child({ message }) {
    return <p>{message}</p>;
}
```

Este mecanismo:

- Hace el flujo de datos explícito

- Facilita el razonamiento sobre la UI

Child → parent (callbacks)

Un componente hijo no modifica directamente el estado del padre.

Para comunicar información hacia arriba:

- El padre pasa una función como prop
- El hijo ejecuta esa función cuando ocurre un evento

Ejemplo:

```
function Parent() {  
  function handleClick() {  
    console.log("Evento desde el hijo");  
  }  
  
  return <Child onAction={handleClick} />;  
}  
  
function Child({ onAction }) {  
  return <button onClick={onAction}>Click</button>;  
}
```

Este patrón mantiene el control del estado en el componente adecuado.

Levantar estado (lifting state up)

Levantar estado significa **mover el estado a un ancestro común** cuando varios componentes necesitan acceder o modificarlo.

Se utiliza cuando:

- Dos o más componentes comparten información
- El estado debe mantenerse sincronizado

Ejemplo conceptual:

- El estado se define en el componente padre
- El valor se pasa por props
- Las funciones de actualización se pasan a los hijos

Este patrón:

- Evita duplicación de estado
- Reduce inconsistencias

- Centraliza la lógica

Estado compartido

El estado compartido es aquel que:

- Afecta a múltiples componentes
- No pertenece claramente a uno solo

Formas comunes de manejarlo:

- Levantar estado al componente más cercano posible
- Usar Context cuando el prop drilling se vuelve excesivo
- Usar soluciones de estado global en aplicaciones grandes

Regla práctica:

- El estado se coloca lo más abajo posible, pero tan arriba como sea necesario

Renderizado condicional y composición

Renderizado condicional

El renderizado condicional permite **mostrar u ocultar partes de la UI** según el estado o las props.

En React, el renderizado condicional se basa en expresiones de JavaScript.

Patrones comunes:

- Operador `&&`, cuando solo se quiere renderizar algo si la condición es verdadera
- Operador ternario, cuando se quiere elegir entre dos alternativas

Ejemplo conceptual:

- Si hay datos, se muestra el contenido
- Si no hay datos, se muestra un mensaje o un loader

Este enfoque mantiene la UI sincronizada con el estado de forma declarativa.

Composición vs. herencia

React prioriza la **composición** por sobre la herencia para reutilizar código.

- La herencia es común en otros paradigmas orientados a objetos
- En React, los componentes se combinan entre sí

Composición significa:

- Construir componentes a partir de otros componentes
- Encapsular comportamientos y UI
- Reutilizar sin acoplar jerarquías rígidas

La herencia rara vez es necesaria en React.

Patrones comunes de composición

Algunos patrones habituales:

- Componentes contenedores que envuelven contenido (`Card`, `Layout`, `Modal`)
- Uso de `children` para definir estructura flexible
- Componentes que reciben otros componentes como props

Ejemplo conceptual:

- Un componente define el marco
- El contenido se inyecta desde afuera

Estos patrones permiten crear UIs flexibles y reutilizables.

Componentes “contenedor” y “presentacional”

Esta distinción es un **concepto arquitectónico**, no una regla estricta.

Componentes presentacionales:

- Se enfocan en la UI
- Reciben datos por props
- Tienen poca o ninguna lógica

Componentes contenedores:

- Manejan estado y lógica
- Conectan con datos
- Pasan props a los presentacionales

Este enfoque:

- Mejora la separación de responsabilidades
- Facilita el testeo
- Hace el código más mantenible

En React moderno, esta separación puede ser más flexible, pero el concepto sigue siendo útil.

Manejo de listas y datos

Renderizado de listas

El renderizado de listas en React se realiza a partir de **arrays de datos**.

- Se transforma un array en elementos JSX
- Se utiliza generalmente `map`
- Cada elemento debe tener una `key` única

Ejemplo:

```
<ul>
  {items.map(item => (
    <li key={item.id}>{item.name}</li>
  ))}
</ul>
```

Este patrón permite que la UI refleje directamente el contenido de los datos.

Filtrado y ordenamiento

El filtrado y el ordenamiento se realizan **antes del render**, usando métodos de array.

Ejemplo conceptual:

- Se filtran los datos según un criterio
- Se ordenan si es necesario
- Se renderiza el resultado

Ejemplo:

```
const visibleItems = items
  .filter(item => item.active)
  .sort((a, b) => a.name.localeCompare(b.name));
```

Luego se renderiza `visibleItems`.

Buenas prácticas:

- No modificar el array original
- Mantener la lógica de transformación simple y legible

Datos asincrónicos

Muchos datos en React provienen de **fuentes asincrónicas**, como APIs.

Características habituales:

- Los datos no están disponibles en el primer render
- Se cargan luego del montaje del componente
- Suelen gestionarse con `useEffect`

Modelo mental:

- Render inicial sin datos
- Inicio de la carga
- Actualización del estado al recibir la respuesta

Loading y error states

Cuando se trabajan datos asíncronos, se suelen manejar estados explícitos.

Estados comunes:

- Loading: indica que los datos se están cargando
- Error: indica que ocurrió un problema
- Success: indica que los datos están disponibles

Ejemplo conceptual:

```
if (loading) return <Spinner />;  
if (error) return <ErrorMessage />;  
  
return <DataList items={data} />;
```

Este patrón:

- Hace la UI predecible
- Evita renderizados inconsistentes
- Mejora la experiencia de usuario

Fetching y asincronía

fetch / axios

Para obtener datos desde servidores externos se utilizan **peticiones HTTP**.

En React, esto se realiza normalmente usando:

- `fetch` (API nativa del navegador)
- Librerías como `axios`

Ambas opciones permiten:

- Realizar requests GET, POST, PUT, DELETE
- Trabajar con promesas
- Manejar respuestas y errores

La elección entre `fetch` y `axios` suele depender de preferencias y necesidades del proyecto.

Peticiones en useEffect

Las peticiones asíncronas suelen iniciarse dentro de `useEffect`.

Modelo típico:

- El componente se monta
- Se ejecuta el efecto
- Se realiza la petición
- Se actualiza el estado con la respuesta

Ejemplo conceptual:

```
useEffect(() => {  
  fetchData();  
}, []);
```

Este patrón evita ejecutar la petición en cada render.

Cancelación de requests

En algunos casos, una petición puede seguir activa cuando el componente se desmonta.

Esto puede provocar:

- Warnings
- Actualizaciones de estado innecesarias

Para evitarlo:

- Se puede usar `AbortController` con `fetch`
- Se cancela la petición en la función de cleanup del efecto

Modelo conceptual:

- Se inicia la petición
- Se retorna una función de limpieza
- La limpieza cancela la petición si el componente se desmonta

Estados de carga

Al trabajar con asincronía, se suelen manejar estados explícitos.

Estados habituales:

- Loading: indica que la petición está en curso
- Error: indica que ocurrió un fallo
- Data: contiene los datos obtenidos

Este enfoque:

- Hace el flujo de la UI más claro
- Evita renderizados intermedios confusos
- Mejora la experiencia de usuario

Buenas prácticas básicas

Buenas prácticas comunes:

- No iniciar peticiones directamente durante el render
- Manejar siempre loading y error states
- Cancelar peticiones cuando sea necesario
- Evitar lógica compleja dentro de `useEffect`
- Separar la lógica de fetching de la UI cuando crece

Estas prácticas ayudan a mantener componentes más simples y predecibles.

Hooks adicionales más usados

useRef

`useRef` permite crear una referencia mutable que se mantiene entre renders sin provocar re-render cuando cambia.

Usos típicos:

- Acceder directamente a un elemento del DOM.
- Almacenar valores mutables que no afectan al renderizado.

```
import { useRef } from "react";

function InputFocus() {
  const inputRef = useRef(null);

  const focusInput = () => {
```

```

        inputRef.current.focus();
    };

    return (
        <>
            <input ref={inputRef} />
            <button onClick={focusInput}>Focus</button>
        </>
    );
}

```

La propiedad `.current` puede modificarse libremente sin disparar un nuevo render.

useMemo

`useMemo` permite memorizar el resultado de un cálculo costoso y recomputarlo solo cuando cambian sus dependencias.

Se utiliza para:

- Evitar cálculos innecesarios en cada render.
- Mantener estabilidad de valores derivados complejos.

```

import { useMemo } from "react";

const total = useMemo(() => {
    return items.reduce((acc, item) => acc + item.price, 0);
}, [items]);

```

Si las dependencias no cambian, se reutiliza el valor memorizado.

useCallback

`useCallback` memoriza una función para evitar que se recree en cada render.

Es especialmente útil cuando:

- La función se pasa como prop a componentes hijos.
- Se usa como dependencia de otros hooks.

```

import { useCallback } from "react";

const handleClick = useCallback(() => {
    setCount(c => c + 1);
}, []);

```

`useCallback` es conceptualmente equivalente a `useMemo` aplicado a funciones.

Cuándo NO usarlos

No se recomienda usar estos hooks por defecto.

Casos en los que no aportan valor:

- Cálculos triviales o rápidos.
- Componentes simples sin problemas de performance.
- Funciones que no se pasan a hijos ni a dependencias.

El uso excesivo puede:

- Dificultar la lectura del código.
- Introducir dependencias incorrectas.
- Generar optimizaciones innecesarias.

Context y estado global

Context API

La Context API permite compartir datos entre componentes sin necesidad de pasar props manualmente a través de múltiples niveles del árbol.

Se utiliza para valores que son necesarios en muchas partes de la aplicación, como:

- Tema visual (theme).
- Usuario autenticado.
- Configuración global.

Context no reemplaza al estado local, sino que complementa su uso.

Cuándo usar context

Se recomienda usar context cuando:

- Un mismo dato se consume en muchos componentes no relacionados directamente.
- El *prop drilling* comienza a afectar la legibilidad del código.

No se recomienda usar context para:

- Estado local de componentes.
- Datos que cambian muy frecuentemente y afectan grandes porciones del árbol.

Context no es una solución universal para manejo de estado.

Provider y consumer

Un context se crea con `createContext` y se expone mediante un `Provider`.

```
import { createContext } from "react";

const ThemeContext = createContext();
```

El `Provider` define el valor que se comparte:

```
<ThemeContext.Provider value="dark">
  <App />
</ThemeContext.Provider>
```

El valor se consume usando `useContext`:

```
import { useContext } from "react";

const theme = useContext(ThemeContext);
```

El componente consumidor debe estar dentro del árbol del `Provider`.

Problemas de performance

Cuando el valor del `Provider` cambia, todos los componentes consumidores se re-renderizan.

Problemas comunes:

- Contexts con valores que cambian muy seguido.
- Providers que envuelven todo el árbol de la aplicación sin necesidad.

Buenas prácticas:

- Separar contexts por responsabilidad.
- Memorizar valores del provider cuando corresponde.
- Evitar colocar estado altamente volátil en context.

Alternativas

Para aplicaciones más complejas, existen alternativas al Context API:

- Librerías de estado global (Redux, Zustand, Jotai, Recoil).
- Manejo de estado del servidor (React Query / TanStack Query, SWR).

Estas herramientas ofrecen:

- Mejor control de actualizaciones.
- Separación más clara entre estado local, global y remoto.
- Patrones más escalables para aplicaciones grandes.

La elección depende del tamaño de la aplicación y de la complejidad del flujo de datos.

Estilos en React

CSS tradicional

Se pueden usar hojas de estilo CSS tradicionales importándolas en los componentes.

```
import './styles.css';
```

Las clases se aplican usando `className`.

```
<div className="container">Contenido</div>
```

Este enfoque es simple y familiar, pero puede generar colisiones de nombres en aplicaciones grandes.

CSS Modules

CSS Modules permiten encapsular estilos a nivel de componente.

El archivo se nombra con la convención `.module.css`:

```
import styles from './Button.module.css';
```

```
<button className={styles.primary}>Click</button>
```

Las clases se transforman en identificadores únicos, evitando conflictos globales.

Styled-components

Styled-components es una librería de CSS-in-JS que permite definir estilos como componentes.

Ejemplo conceptual:

```
const Button = styled.button`
  background: black;
  color: white;
`;
```

Este enfoque:

- Encapsula estilos y lógica.
- Permite estilos dinámicos basados en props.
- Introduce una dependencia externa y mayor abstracción.

Inline styles

Los estilos inline se definen como objetos JavaScript.

```
<div style={{ color: "red", fontSize: "16px" }} />
```

Características:

- Las propiedades usan camelCase.
- No se soportan pseudoclases ni media queries.
- Útiles para estilos dinámicos simples.

Convenciones comunes

Prácticas habituales al trabajar con estilos en React:

- Preferir encapsulación de estilos por componente.
- Evitar estilos globales innecesarios.
- Mantener una convención consistente en nombres y estructura.
- Separar estilos estáticos de estilos dinámicos.

La elección del enfoque depende del tamaño del proyecto y del equipo.

Routing

React, por sí solo:

- renderiza componentes
- maneja estado
- **no maneja URLs**

El routing se vuelve necesario **solo si la app tiene más de una vista lógica**.

Usar routing cuando:

- hay **más de una pantalla**
- la URL importa (compartir links, refrescar página)
- hay secciones claras:
 - `/login`
 - `/usuarios`
- la app crece más allá de una sola vista

Ejemplos:

- dashboards
- paneles de administración
- apps con login
- apps con navegación lateral

No usar routing cuando:

- la app es una sola pantalla
- solo se muestran/ocultan componentes
- el flujo es lineal
- la URL no aporta valor

Ejemplos:

- formularios simples
- pequeñas herramientas internas
- prototipos rápidos

React Router es la **librería** más utilizada para manejar navegación en aplicaciones React.

Permite:

- Asociar URLs a componentes.
- Navegar sin recargar la página.
- Mantener el estado de la aplicación en una SPA.

El routing se maneja del lado del cliente (client-side routing).

Rutas

Las rutas se definen declarativamente usando componentes provistos por React Router.

Ejemplo básico:

```
import { BrowserRouter, Routes, Route } from
"react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Cada ruta asocia un `path` con un componente a renderizar.

Links

La navegación entre rutas se realiza usando `Link` o `NavLink`, en lugar de etiquetas `<a>`.

```
import { Link } from "react-router-dom";

<Link to="/about">About</Link>
```

Esto evita recargas completas de la página y mantiene el estado de la aplicación.

`NavLink` permite aplicar estilos según la ruta activa.

Params

Las rutas pueden definir parámetros dinámicos.

```
<Route path="/users/:id" element={<User />} />
```

El parámetro se obtiene usando `useParams`:

```
import { useParams } from "react-router-dom";

const { id } = useParams();
```

Los params suelen usarse para:

- Identificadores de recursos.
- Detalles de elementos individuales.

Rutas protegidas

Las rutas protegidas restringen el acceso según alguna condición, como autenticación.

La idea general consiste en:

- Verificar el estado de autenticación.
- Renderizar la ruta o redirigir según corresponda.

Ejemplo conceptual:

```
function ProtectedRoute({ children }) {
  return isAuthenticated ? children : <Navigate to="/login" />;
}
```

Este patrón permite centralizar la lógica de acceso y mantener las rutas declarativas.

Buenas prácticas y patrones

→ Componentes pequeños

Se recomienda crear componentes pequeños y enfocados en una única responsabilidad.

Beneficios:

- Mayor legibilidad.
- Reutilización más sencilla.
- Menor complejidad cognitiva.

Un componente grande suele ser señal de que puede dividirse en varios más simples.

→ Separación de responsabilidades

Cada componente debería encargarse de una tarea clara:

- Presentación (UI).
- Lógica de estado.
- Obtención de datos.

Separar responsabilidades facilita el mantenimiento y el testeo.

→ Evitar re-renders innecesarios

Los re-renders son normales en React, pero pueden volverse costosos.

Buenas prácticas:

- Evitar pasar objetos o funciones nuevas innecesariamente.
- Memorizar valores o callbacks solo cuando hay un problema real.
- Mantener el estado lo más local posible.

La optimización prematura suele generar más problemas que soluciones.

→ Organización de carpetas

No existe una única estructura correcta.

Convenciones comunes:

- Agrupar por funcionalidad o feature.
- Colocar componentes, estilos y tests relacionados juntos.
- Evitar carpetas genéricas excesivamente grandes.

La consistencia dentro del proyecto es más importante que la estructura elegida.

Convenciones para nombres

Usar nombres claros y descriptivos mejora la comprensión del código.

Convenciones habituales:

- Componentes en **PascalCase**.
- Hooks personalizados con prefijo `use`.
- Nombres que expresen intención, no implementación.

Un buen nombre reduce la necesidad de comentarios adicionales.

Performance

Memoización

La memoización permite evitar cálculos o renders innecesarios cuando los datos no cambian.

En React se utiliza principalmente mediante:

- `useMemo` para valores derivados.
- `useCallback` para funciones.
- `React.memo` para componentes.

La memoización debe aplicarse solo cuando existe un problema real de performance.

Lazy loading

El lazy loading permite cargar componentes de forma diferida, solo cuando se necesitan.

Se utiliza para:

- Reducir el tamaño del bundle inicial.
- Mejorar el tiempo de carga inicial.

En React se implementa con `React.lazy` y `Suspense`.

Code splitting

El code splitting divide el código en múltiples bundles más pequeños.

Beneficios:

- Descargas más rápidas.
- Mejor experiencia en aplicaciones grandes.

El lazy loading de rutas es una forma común de code splitting.

Cuándo optimizar

No se recomienda optimizar performance de forma prematura.

Se justifica preocuparse por performance cuando:

- La UI se siente lenta o poco responsiva.
- Existen renders costosos medidos o percibidos.
- La aplicación crece en complejidad.

En aplicaciones pequeñas o medianas, la claridad del código suele ser más importante que micro-optimizaciones.

Errores comunes y debugging

Infinite re-renders

Un re-render infinito ocurre cuando un cambio de estado provoca inmediatamente otro cambio de estado.

Causas frecuentes:

- Llamar a un setter de estado durante el render.
- Efectos sin dependencias correctas.

Ejemplo problemático:

```
setCount(count + 1);
```

Los cambios de estado deben ejecutarse en handlers o efectos bien definidos.

Dependencias mal definidas

Errores en el array de dependencias de `useEffect` pueden generar:

- Efectos que se ejecutan demasiadas veces.
- Efectos que no se ejecutan cuando deberían.

Buenas prácticas:

- Incluir todas las variables usadas dentro del efecto.
- Confiar en las advertencias del linter de hooks.

Keys incorrectas

Usar keys incorrectas en listas puede causar problemas de renderizado.

Errores comunes:

- Usar el índice del array como key.
- Usar valores no estables.

Las keys deben ser:

- Únicas.
- Estables entre renders.

Mutación de estado

Modificar directamente el estado rompe el modelo de React.

Ejemplo incorrecto:

```
state.push(item);  
setState(state);
```

Siempre se debe crear una nueva referencia al actualizar estado.

Uso incorrecto de hooks

Los hooks deben respetar sus reglas.

Errores frecuentes:

- Llamarlos dentro de condicionales o bucles.
- Usarlos fuera de componentes o hooks personalizados.

Estas situaciones suelen generar errores difíciles de rastrear y comportamientos inesperados.

Ecosistema React

React vs. frameworks (Next, Remix)

React es una **librería** enfocada en la construcción de interfaces de usuario.

Frameworks como Next.js o Remix se construyen sobre React y agregan:

- Routing integrado.
- Renderizado del lado del servidor (SSR).
- Generación de sitios estáticos (SSG).
- Convenciones de estructura y tooling.

React ofrece flexibilidad; los frameworks ofrecen estructura y decisiones predefinidas.

Estado global (Redux, Zustand)

Para manejar estado compartido a gran escala existen librerías especializadas.

Redux:

- Enfoque más estructurado.
- Flujo de datos explícito.
- Mayor verbosidad.

Zustand:

- API más simple.
- Menos boilerplate.
- Uso más cercano al estado local.

La elección depende de la complejidad del estado y del equipo.

UI libraries

Las librerías de UI proveen componentes visuales reutilizables.

Ejemplos comunes:

- Material UI.
- Chakra UI.
- Ant Design.
- shadcn/ui.

Estas librerías aceleran el desarrollo, pero imponen decisiones de diseño y estructura.

Herramientas habituales

En el ecosistema React suelen aparecer herramientas complementarias:

- Manejo de datos remotos (TanStack Query, SWR).
- Routing (React Router).
- Testing (Testing Library, Jest, Vitest).
- Linting y formato (ESLint, Prettier).

Este conjunto forma un mapa mental del stack típico alrededor de React.

Anexo: Guía práctica para crear una aplicación en React

A continuación se listan los pasos básicos habituales para crear una aplicación React genérica, partiendo desde un entorno vacío y usando VSCode como editor.

Crear el proyecto

1. Abrir una **terminal**.
 - a. Desde el menú Terminal, seleccionar New Terminal.
 - b. Con el atajo de teclado: `Ctrl + ñ / Ctrl + Shift + ñ`.
 - c. `Ctrl + Shift + P`, escribir *Terminal: New Terminal* y `Enter`.
2. Ubicarse en la **carpeta** donde se quiera crear el proyecto.
3. Ejecutar el comando para crear un proyecto con **Vite**:
`npm create vite@latest`
4. Ingresar el **nombre** del proyecto.
5. Seleccionar **React** como **framework**.
6. Seleccionar **JavaScript** o **TypeScript** según preferencia.
7. Entrar a la **carpeta** del proyecto:
`cd nombre-del-proyecto`
8. Instalar las **dependencias** que se vayan a utilizar:
`npm install dependencia`
Ejemplos habituales:
`npm install react-router-dom` (para **Routing**)
`npm install axios` (alternativa a **fetch**)
9. **Abrir** el proyecto en VSCode:
`code .` (el punto abre la carpeta actual)

Si el comando `code` no funciona, suele ser porque VSCode no está agregado al PATH (se puede configurar desde VSCode).

Ejecutar el proyecto

1. Ejecutar el **servidor** de desarrollo:
`npm run dev`
Con esto:
 - a. Se levanta un servidor de desarrollo local (Vite).
 - b. La app se sirve normalmente en `http://localhost:5173`.
 - c. El navegador puede cargar la aplicación y ver los cambios en tiempo real cuando se guarda un archivo (*hot reload*).
2. Abrir el **navegador** en la URL indicada por la terminal.
3. **Verificar** que la aplicación inicial se renderiza correctamente.

Entender la estructura inicial

Identificar los **archivos** principales:

- `index.html`: punto de entrada HTML.
- `src/main.jsx`: inicialización de React y montaje del árbol.
- `src/App.jsx`: componente principal de la aplicación.
- `src/assets/`: recursos estáticos.

No modificar `index.html` salvo necesidad específica.

Limpiar el proyecto base

1. **Abrir** `App.jsx`.
2. **Eliminar** contenido de ejemplo innecesario.
3. Dejar una **estructura** mínima:

```
function App() {  
  return (  
    <div>  
      <h1>App</h1>  
    </div>  
  );  
}  
  
export default App;
```

4. Eliminar **estilos y assets** que no se utilicen.

Crear estructura de carpetas básica

Dentro de `src/`, crear **carpetas** según convención:

- `components/`: componentes reutilizables.
- `pages/`: vistas o pantallas.
- `services/`: lógica de fetching o APIs.
- `hooks/`: hooks personalizados.
- `styles/`: estilos globales o compartidos.

Ajustar la estructura según el tamaño del proyecto.

Crear un componente simple

1. Crear un **archivo** en `components/`.

Ejemplo: `Button.jsx`.

2. Escribir un **componente funcional** con sus props.

Ejemplo: botón con props `label` y `onClick`

```
function Button({ label, onClick }) {  
  return <button onClick={onClick}>{label}</button>;  
}  
  
export default Button;
```

3. **Importar** el componente en `App.jsx` (arriba del archivo).

Ejemplo:

```
import Button from "../components/Button";
```

Notas:

- a. `../components/Button` es la **ruta relativa** desde `App.jsx`.
- b. No se pone `.jsx` (Vite lo interpreta solo).
- c. Se usa `import X from` porque el componente se exportó como `default`.

4. **Usar** el componente en `App.jsx`.

Ejemplo:

```
function App() {  
  return (  
    <div>  
      <Button label="Click acá" onClick={handler} />  
    </div>  
  );  
}
```

Notas:

- a. `<Button />` se usa como si fuera una etiqueta HTML propia.
- b. Las **props** se pasan como **atributos** (pasar datos y funciones a componentes hijos mediante props).
- c. React reemplaza `<Button />` por lo que ese componente retorna.

Usar props y estado

1. **Importar** `useState` en el componente necesario.

```
import { useState } from "react";
```

Notas:

- a. `useState` es un **hook** que viene de React, así que se importa desde `"react"`.
- b. Se usan llaves `{}` porque **no es export default**, sino un export nombrado.
- c. Solo se importa en los componentes que realmente lo usan.

2. **Declarar** estado local en el mismo componente:

```
const [valor, setValor] = useState(valorInicial);
```

Ejemplo: contador

```
const [count, setCount] = useState(0);
```

3. Actualizar estado mediante **handlers**:

- a. definir una función (handler),
- b. que se ejecute ante un evento (click, submit, change),
- c. y dentro de esa función llamar a **setEstado**.

Ejemplo completo: componente hijo botón contador en el componente padre

App.jsx

```
import { useState } from "react";
import Button from "../components/Button";

function App() {
  const [count, setCount] = useState(0);

  function handleIncrement() {
    setCount(count + 1);
  }

  return (
    <div>
      <p>Contador: {count}</p>
      <Button label="Sumar" onClick={handleIncrement} />
    </div>
  );
}

export default App;
```

Nota: Sin `export default App`; al final del código, React no puede arrancar la app desde `main.jsx`.

Manejar eventos

1. Definir handlers como funciones.
2. Asociar eventos usando props JSX:
`<button onClick={handleClick}>Click</button>`
3. Evitar ejecutar funciones directamente durante el render.

Renderizar listas

1. Crear el **componente hijo** `Item.jsx`:

```
function Item({ data }) {  
  return <li>{data.text}</li>;  
}  
  
export default Item;
```

2. **Importar** `useState` y el componente hijo en el componente **padre**.

```
import { useState } from "react";  
import Item from "../Item";
```

3. Guardar datos en un **array** en el componente **padre**. Usar **keys** únicas y estables.
Ejemplo:

```
const [items, setItems] = useState([  
  { id: 1, text: "Elemento 1" },  
  { id: 2, text: "Elemento 2" },  
  { id: 3, text: "Elemento 3" },  
]);
```

4. Usar `map` para **renderizar** los elementos en el componente **padre**.

Ejemplo completo en `App.jsx`:

```
import { useEffect, useState } from "react";  
import { fetchUsers } from "../services/usersService";  
  
function App() {  
  const [users, setUsers] = useState([]);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(null);  
  
  useEffect(() => {  
    setLoading(true);  
  
    fetchUsers()  
      .then((data) => {  
        setUsers(data);  
        setError(null);  
      })  
      .catch((err) => {  
        setError(err.message);  
      })  
      .finally(() => {  
        setLoading(false);  
      });  
  });  
}
```

```

    }, []);

    if (loading) {
      return <p>Cargando...</p>;
    }

    if (error) {
      return <p>Error: {error}</p>;
    }

    return (
      <ul>
        {users.map((user) => (
          <li key={user.id}>{user.name}</li>
        ))}
      </ul>
    );
  }

  export default App;

```

Fetching de datos

1. **Crear** una función de fetching en `services/` (no hacer fetch directamente en el render).

Ejemplo: archivo `userService.js` para pedir datos de usuarios

```

export async function fetchUsers() {
  const response = await
  fetch("https://api.example.com/users");

  if (!response.ok) {
    throw new Error("Error al obtener usuarios");
  }

  const data = await response.json();
  return data;
}

```

Nota: El `if` maneja el error HTTP.

2. Usar `useEffect` para **ejecutar** la petición en el **componente padre**.

Ejemplo:

```

import { useEffect, useState } from "react";

```

```
import { fetchUsers } from "../services/usersService";

function App() {
  useEffect(() => {
    fetchUsers();
  }, []);

  return <div>App</div>;
}
```

Nota: El `[]` significa “Ejecutar esto solo cuando el componente se monta”.

3. Manejar estados de `loading`, `error` y `data` en el **componente padre**.

Ejemplo completo en `App.jsx`:

```
import { useEffect, useState } from "react";
import { fetchUsers } from "../services/usersService";

function App() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);

    fetchUsers()
      .then((data) => {
        setUsers(data);
        setError(null);
      })
      .catch((err) => {
        setError(err.message);
      })
      .finally(() => {
        setLoading(false);
      });
  }, []);

  if (loading) {
    return <p>Cargando...</p>;
  }

  if (error) {
    return <p>Error: {error}</p>;
  }

  return <div>
    {users.map((user) => <div>{user}</div>)}
  </div>;
}
```



```

        return <p>Error: {error}</p>;
    }

    return (
        <ul>
            {users.map((user) => (
                <li key={user.id}>{user.name}</li>
            ))}
        </ul>
    );
}

export default App;

```

Manejar estilos

1. Elegir **estrategia** de estilos (CSS tradicional, CSS Modules, etc.), y mantener consistencia en convenciones.
2. Crear o guardar el **archivo** `.css` a utilizar dentro de la carpeta `components/`.
3. **Importar** estilos en el componente correspondiente (el que los usa).
 - a. Para CSS tradicional:


```
import './styles.css';
```
 - b. Para CSS Modules:


```
import styles from './styles.module.css';
```
4. Aplicar **clases** usando `className` (la palabra reservada de JSX).
 - a. Para CSS tradicional, los nombres de clase son strings normales:


```
className="button"
```
 - b. Para CSS Modules, las clases son propiedades del objeto styles:


```
className={styles.button}
```
5. Para clases (estilos) **condicionales**:
 - a. CSS tradicional:


```
className={`button ${isActive ? "button-active" : ""}`} `
```
 - b. CSS Modules:


```
className={`${styles.button} ${ isActive ? styles.active : ""}`} `
```

Agregar routing

1. **Instalar** React Router desde la **terminal**:


```
npm install react-router-dom
```

2. Importar BrowserRouter en main.jsx.

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  (
    <BrowserRouter>
      <App />
    </BrowserRouter>
  );
);
```

3. Importar Routes y Route en App.jsx y definir rutas.

```
import { Routes, Route } from "react-router-dom";
import Home from "./pages/Home";
import About from "./pages/About";
import NotFound from "./pages/NotFound";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="*" element={<NotFound />} />
    </Routes>
  );
}

export default App;
```

Nota: El * es la ruta 404 por defecto.

4. Navegar usando Link.

Ejemplo: Navbar.jsx:

```
import { Link } from "react-router-dom";

function Navbar() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>
    </nav>
  );
}
```

```
    );  
  }  
  
  export default Navbar;
```

Verificar y ordenar

1. Revisar estructura de **carpetas**.
2. Renombrar componentes y archivos para mayor claridad.
3. Eliminar código no usado.
4. Confirmar que la aplicación **corre** sin errores.